A Proposal: An Anonymous Identification Verification System

Based on Ring Signatures with Isolated Smart Card Hardware

Dinh Anh Khoa Nguyen - 245849840 - nguy9840@mylaurier.ca
Jiangtao Wang - 245823800 - Wang2370@mylaurier.ca
Leila Chen - 245850620 - chen5062@mylaurier.ca
Yuyang Wu - 245817310 - wuxx7310@mylaurier.ca

January 23, 2026

Abstract

This proposal outlines the engineering implementation of a privacy-preserving messaging platform that decouples message integrity from identity revelation. Leveraging Yubikey hardware PRF and nested Ring Signatures, the system allows users to send undeniable, tamper-proof messages anonymously, with the exclusive capability to retroactively prove authorship. The project adopts an Agile methodology, utilizing React/TypeScript for the frontend and Node.js for the backend.

Contents

1 Executive Summary and Highlights **3**
1.1 Overview . . . 3
1.2 Key Highlights . . . 3

2 Problem Motivation **4**
2.1 Problem Statement . . . 4
2.2 Objectives . . . 4

3 System Overview and Workflow **4**
3.1 System Roles and Trust Assumptions . . . 4
3.2 Workflow . . . 4

4 Methodology and Technical Design **5**
4.1 Security Model . . . 5
4.2 Technical Stack Chosen . . . 5
4.3 Cryptographic Construction . . . 5

5 System Design **6**
5.1 Architecture Overview . . . 6
5.2 Component Diagram . . . 6
5.3 Data Flow . . . 6

6 Genesis Sprint and Milestones **6**
6.1 Sprint 0: Feasibility & PoC (2 Weeks) . . . 6
6.2 Milestone 1: Core Cryptography Module (2 weeks) . . . 7
6.3 Milestone 2: Backend & Basic UI (2 weeks) . . . 7
6.4 Milestone 3: Integration & The "Reveal" Flow (2 weeks) . . . 7
6.5 Milestone 4: Evaluation and Documentation (1 week) . . . 8

1 Executive Summary and Highlights

1.1 Overview

Project AIVS (Anonymous Identification Verification System) is a cryptographic engineering initiative designed to solve the a paradox in secure communications: the trade-off between anonymity and accountability, and introducing a hardware smartcard to participate in this process to ensure the security of the private keys.

1.2 Key Highlights

- **Hardware Security:** Utilizes Yubikey PRF to generate unpredictable passphrases, ensuring keys never leave the hardware boundary.
- **Nested Ring Signatures:** Employs a dual-layer signing mechanism (Inner Ring for identity proof, Outer Ring for anonymous transmission) to ensure mathematical unlinkability.
- **Selective Disclosure:** Grants the sender full control over identity revelation. No central authority can unmask the user; only the holder of the physical hardware token can decrypt the proof.
- **Agile Engineering Stack:** Built on a modern, type-safe stack (React + TypeScript + Node.js) designed for auditability and rapid iteration.

2 Problem Motivation

2.1 Problem Statement

In current cryptographic systems, users typically face a binary choice:

1. **Signed Messages:** Authenticity is guaranteed, but privacy is compromised immediately (e.g., PGP).
2. **Anonymous Messages:** Privacy is maintained, but credibility is low, and the author cannot later claim credit or reputation for the content (e.g., Tor-based drops).

There is a lack of infrastructure that supports *conditional anonymity*: messages are anonymous by default, yet authorship can be proven later by the sender alone. Where a user can speak freely now and prove they spoke later, without relying on a trusted third party.

2.2 Objectives

- **Objective A:** Anonymous message dispatching where the sender is mathematically hidden among a set of 7 decoys (1-of-8 Ring Signature).
- **Objective B:** Integrity verification by the recipient without identity knowledge.
- **Objective C:** A "Challenge-Response" flow where the sender can prove authorship by unlocking a cryptographic proof using their physical Yubikey.

3 System Overview and Workflow

3.1 System Roles and Trust Assumptions

The system involves three primary roles: the Sender, the Recipient, and a Backend Server, with a YubiKey hardware token participating in cryptographic operations. Both the Sender and the Recipient interact with the system through a web browser, while the backend server functions solely as a message relay and storage service.

All cryptographic operations are performed locally on the client side with the assistance of the YubiKey. The server does not generate, store, or process private keys, and only handles opaque data blobs, including messages, signatures, and encrypted proofs. The YubiKey is used to provide hardware-bound PRF outputs that participate in one-time key derivation and proof encryption, without exposing secret material to the browser or operating system.

The system does not maintain a global signing key registry. Each message is associated with a freshly derived one-time key, and no signing keys are reused across messages.

3.2 Workflow

The workflow consists of an anonymous message transmission phase followed by an optional identity verification phase.

During message transmission, the Sender composes a message using a browser-based client with a connected YubiKey. A one-time key is derived with the participation of the YubiKey and used, together with a fixed set of randomly generated decoy keys, to construct a ring signature over the message. To support future authorship verification, an additional inner ring signature is generated and encrypted as a proof using a symmetric key derived from the YubiKey PRF output. An outer

ring signature is then computed over the message concatenated with the hash of the encrypted proof. The message, outer signature, and encrypted proof are transmitted to the Recipient via the backend server.

Upon receipt, the Recipient verifies the outer ring signature to confirm message integrity and authenticity, while the Sender's identity remains hidden within the anonymity set. The encrypted proof is not required for this initial verification.

If identity verification is later requested, the Recipient sends the original message, signature, and encrypted proof to the claimed Sender. Using the same YubiKey, the Sender derives the corresponding decryption key and decrypts the proof to reveal the inner ring signature. The decrypted signature is returned to the Recipient, who verifies its validity and linkability against the original message and signature to confirm authorship.

4 Methodology and Technical Design

4.1 Security Model

- **Key Derivation:** The OTK is derived with the participation of the Yubikey, ensuring it cannot be predicted or extracted.
- **Anonymity Set:** 1 real signer + 7 random decoys = 8 total keys. Probability of guessing the signer is 1/8 per message (statistically negligible for identification).
- **Forward Secrecy:** Compromise of the Yubikey allows decryption of past Proofs but does not reveal the identity of past messages *unless* the attacker actively performs the decryption and linking.

4.2 Technical Stack Chosen

- **Frontend:** React, TypeScript.
- **Cryptography (Client):** WebAuthn API (PRF extension), Elliptic Curve Library (e.g., `elliptic` or WASM-based Rust bindings) for Ring Signatures.
- **Backend:** Node.js (Express/NestJS).
- **Database:** PostgreSQL(Stores only blobs: Message, Public Keys Ring, Signature, Encrypted Proof).

4.3 Cryptographic Construction

1. **Setup:** Generate One-Time Key (OTK) via Yubikey. Generate 7 Fake Keys ($FK_{1...7}$).
2. **Inner Layer (The Proof):**

$$Sig_{inner} = RingSign(Message, \{OTK, FK_{1...7}\})$$

$$Key_{proof} = KDF(PRF_{Yubikey}(Hash(OTK)))$$

$$Proof = Encrypt_{AES-GCM}(Sig_{inner}, Key_{proof})$$

where the hash of the OTK is used as a salt to bind the proof key to the one-time signing key.

3. **Outer Layer (The Transmission):**

$$Sig_{outer} = RingSign(Message||Hash(Sig_{inner}), \{OTK, FK_{1...7}\})$$

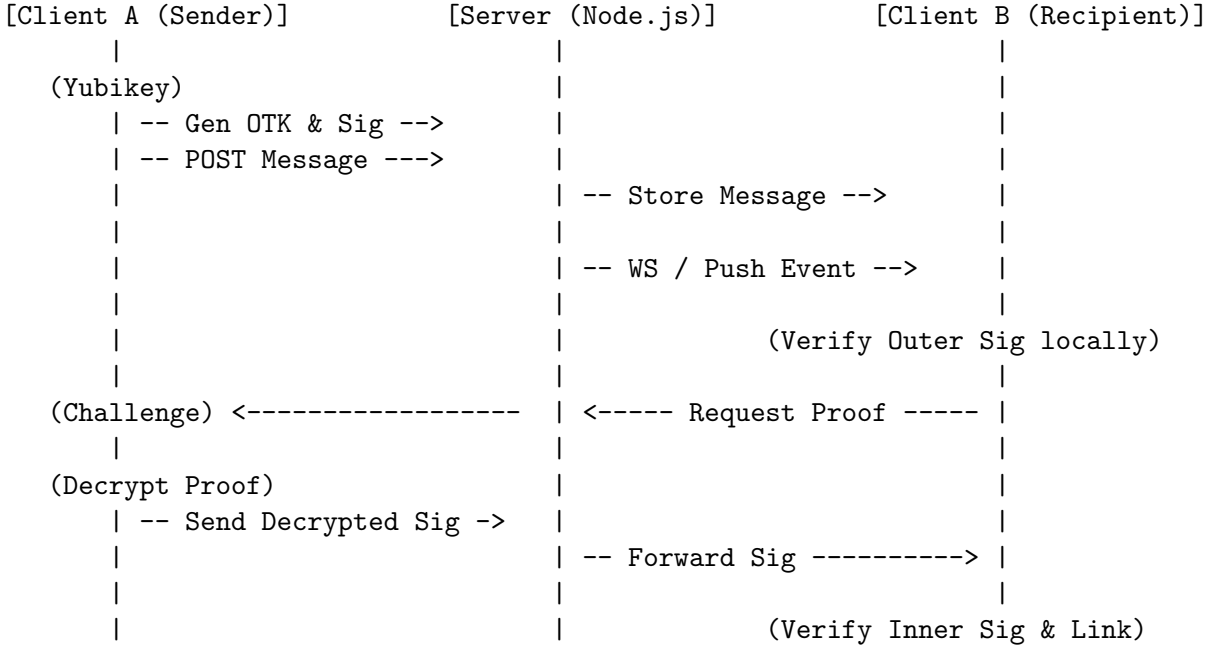
4. **Verification:** Upon decryption, the Linkable Tag of Sig_{inner} must equal the Linkable Tag of Sig_{outer}.

5 System Design

5.1 Architecture Overview

The system follows a **Thick Client, Thin Server** architecture. The server acts purely as a bulletin board and relay, possessing zero knowledge of user identities or private keys, and is explicitly excluded from the trusted computing base.

5.2 Component Diagram



5.3 Data Flow

1. **Registration:** None required for the messaging protocol itself (stateless). Optional username registration maps public handles to WebAuthn Credential IDs for routing.

2. **Signing:** All cryptographic operations occur in the browser memory. The Yubikey is accessed via ‘navigator.credentials.get(prf: ...)‘.

6 Genesis Sprint and Milestones

6.1 Sprint 0: Feasibility & PoC (2 Weeks)

- **Goal:** Validate WebAuthn and Yubikey PRF behavior in the Chrome/Firefox browser.

- **Tasks:**
  - Test WebAuthn registration and authentication process in Chrome/Firefox browser
  - Validate PRF output by using identical salts
  - Derive AES keys from PRF output
- **Deliverable:** A minimal browser-based script that inputs a salt and outputs a consistent AES key using a Yubikey.

6.2 Milestone 1: Core Cryptography Module (2 weeks)

- **Goal:** Implement Linkable Ring Signatures in TypeScript/WASM.
- **Tasks:**
  - Implement a one-time key from WebAuthn
  - Implement linkable ring signatures (1-of-8) in Typescript/WASM
  - Implement linkability tag generation and verification
  - Unit tests for signing, verification, and tag
- **Deliverable:** Unit tests proving 1-of-8 signing and verification, and tag linking.

6.3 Milestone 2: Backend & Basic UI (2 weeks)

- **Goal:** Message transport and basic user interaction.
- **Tasks:**
  - Implement Node.js backend for storing and retrieving message
  - Design message, signature, and proof data format
  - Design and implement a React frontend user interface for composing and viewing messages
  - Integrate WebAuthn for user login
- **Deliverable:** Node.js API to store/retrieve message blobs. React UI to compose messages.

6.4 Milestone 3: Integration & The "Reveal" Flow (2 weeks)

- **Goal:** Connect crypto with UI and demonstrate end-to-end flow.
- **Tasks:**
  - Integrate ring signature into the message
  - Integrate proof encryption and decryption using YubiKey PRF
  - Implement complete end-to-end flow from proof request, decryption, and verification
  - Validate and test the flow
- **Deliverable:** End-to-End flow: Alice sends anonymous msg $\rightarrow$ Bob requests proof $\rightarrow$ Alice reveals $\rightarrow$ Bob verifies.

6.5 Milestone 4: Evaluation and Documentation (1 week)

- **Goal:** Testing the system and prepare for evaluation and document
- **Tasks:**
  - Bug fixes and handle edge-cases if persist
  - Security, threat, and performance review
  - Documentation
- **Deliverable:** A stable prototype with documentation.